Lecture Notes: Energy-Based Models and the Ising Model

Physics of Deep Learning

March 2026

Contents

1 Byers, first module **2**

2 Albert: First Module **7**

2.1 Introduction to Energy-Based Models . . . 7

2.1.1 Goals of Energy-Based Models . . . 7

2.2 The Ising Model . . . 7

2.2.1 The Gibbs Ensemble and Thermodynamics . . . 8

2.2.2 Mean Field Theory and Hubbard-Stratonovich . . . 8

2.2.3 Transitions and Susceptibility . . . 8

2.3 The Hopfield Model . . . 9

2.3.1 Hebbian Learning . . . 9

2.3.2 The Mattis Model . . . 9

3 Byers II Module **10**

3.1 Lecture 5 - Differential geometry in ML . . . 10

3.1.1 Introductory example . . . 10

3.1.2 Differential geometry . . . 11

3.1.3 Recap: ML turns probabilistic problems into geometrical problems . . . 12

3.1.4 Recap: the embedding and the crumpled paper analogy . . . 15

3.1.5 Data sufficiency is a *local* question . . . 16

3.1.6 Locality and data attribution: Neural networks versus Kernel Methods . . . 19

3.1.7 We need to learn how to do *local* PCA . . . 20

Chapter 1

Byers, first module

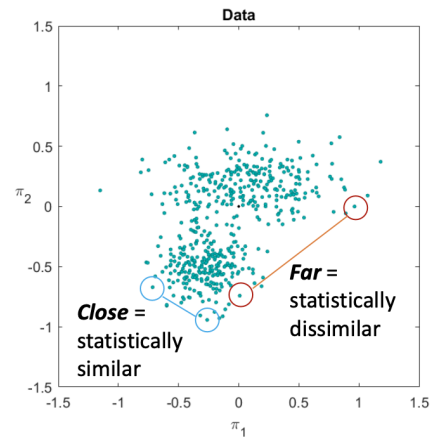
What is Machine Learning?

Create/Learn an Embedding or Feature Vector space that encodes information about *statistical similarity as distances*.

$$-\log p_{\text{similarity}}(y, y') \approx \frac{1}{2}\|\mathbf{y} - \mathbf{y}'\|_2^2$$

How much DATA is enough?

If distant data points are mapped nearby in the model space, then there has to be significant inferential evidence (more data), and likewise if local data points are mapped to large separations by the model.



The Unifying Question

What is machine learning doing geometrically?

Generally, ML methods:

1. Place data into a space.
2. Impose a geometry on that space.
3. Use that geometry to solve a task.
4. The task is encoded as optimizing relative to a loss.

So the real questions are:

- Who chooses the geometry?
- What does the geometry preserve?
- What does the loss value?
- What assumptions are embedded in both?

Figure 1.1: Self-information potential

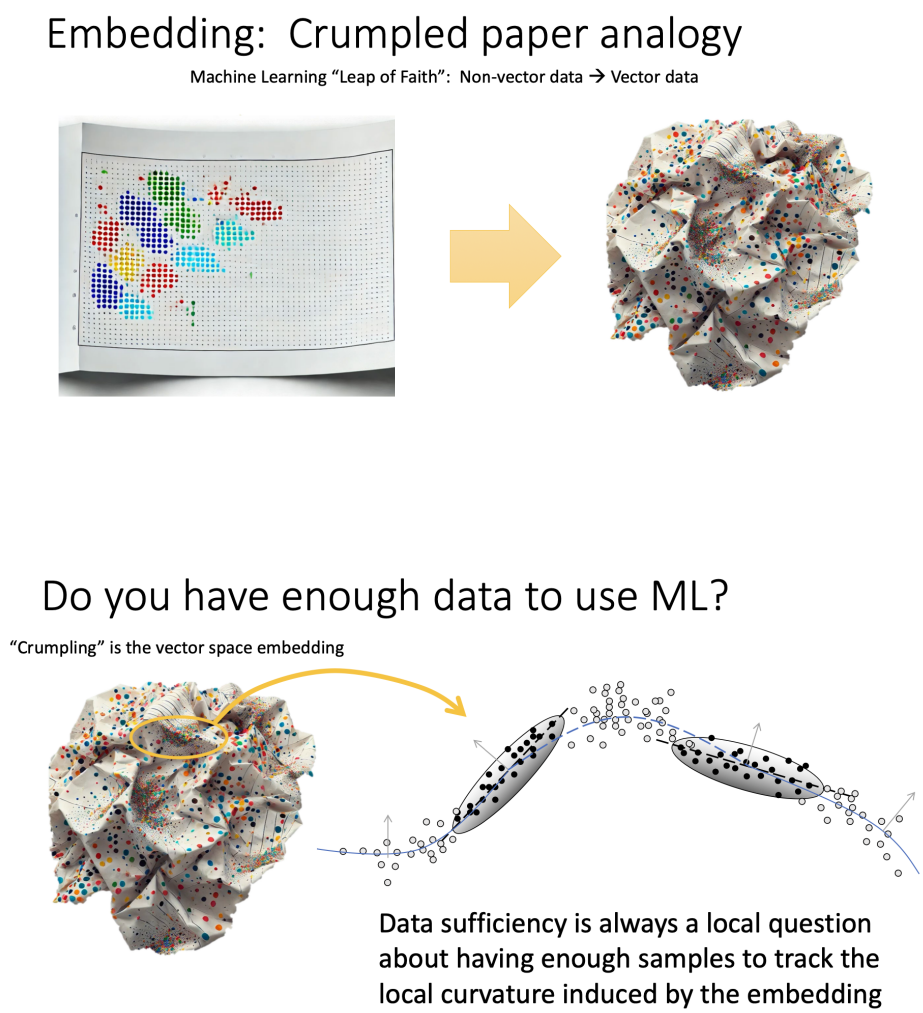

Embedding: Crumpled paper analogy
Machine Learning "Leap of Faith": Non-vector data → Vector data
Do you have enough data to use ML?
"Crumpling" is the vector space embedding
Data sufficiency is always a local question about having enough samples to track the local curvature induced by the embedding

Figure 1.2:

Classical ML: Geometry First, Partition Second

Framework: $f(x) = g(E(x)) = g \circ E(x)$

- E: fixed embedding.
- g: learned partition or function.

Examples:
- PCA + regression.
- Kernel methods.
- Fixed Fourier/wavelet features.
- Local polynomial regression.

Properties:
- Geometry is explicit.
- Partition complexity is measurable.
- Attribution is clear.
- Bias from embedding can be separated from variance in partition.

Philosophically: We choose the coordinate system. Then we learn inside it.

Geometric Partitioning Schemes

A. Cluster-Based (Local Patches)
Assumption:
- Data concentrates in localized regions.

Pros:
- Clear attribution.
- Local uncertainty.
- Honest ignorance in sparse regions.

Cons:
- Complexity grows with volume.
- Weak extrapolation.

B. Low-Dimensional Subspace (Global)
Assumption:
- Data lies near a single linear manifold.

Pros:
- Parameter-efficient.
- Global compression.

Cons:
- Fragile to curvature.
- Diffuse attribution.
- Strong global prior.

C. Hyperplane Partitions
Assumption:
- Data separable by half-spaces.

Pros:
- Compact binary encoding.
- Decision-focused.

Cons:
- Global combinatorial complexity.
- Brittle boundaries.

Local Models	Global Models
Scale with data density	Share parameters globally
Honest uncertainty	Smooth extrapolation
Harder compression	Efficient compression

Question: Is failure to extrapolate a bug or a feature? In science, often a feature.

Figure 1.3: Classical ML: geometry first, partition second

Modern Deep Learning: Learn the Geometry

Deep networks remove the clean separation

$$f(x) = g_\phi\left(E_\theta(x)\right) = g_\phi \circ E_\theta(x)$$

Both E and g are learned jointly adjusting their respective parameters, ϕ and θ, to minimize the loss function, R.

$$\min_\theta \mathrm{E}_{(x,y)\sim\mathcal{D}}\left[R\left(f_\theta(x), y\right)\right]$$

Only the loss function, L, is fixed.
This is a conceptual shift:
We no longer choose the geometry.
We let the loss shape it.

Consequences

- Geometry is task-shaped.
- Distances are not intrinsic.
- Embedding reflects loss, not ontology.
- Attribution becomes entangled.

New option: *Transfer Learning*
Assume that representations, E_θ, learned for one task can apply to a different task.

Surrogate objective function, R → Train to optimize R → Reuse transformation, E, of data to features for data not originally used to co-train g and E but learn a new h:

$$y = h_\phi \circ E_\theta(x)$$

Convolutional Neural Networks

Images
Patches
Pixels

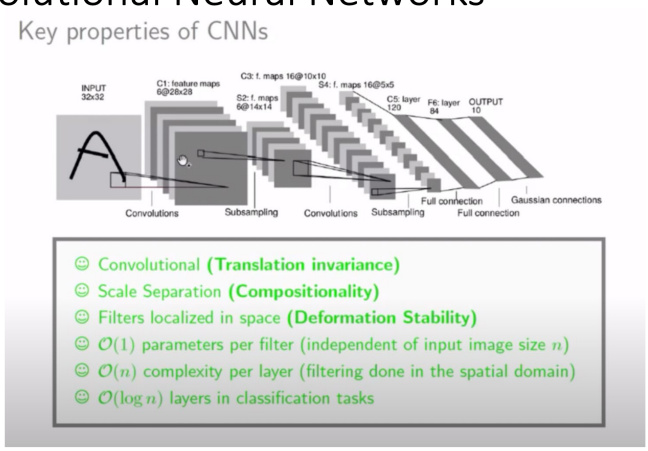


Figure 1.4: Modern ML: learn geometry together with partition

Chapter 2

Albert: First Module

2.1 Introduction to Energy-Based Models

Today, we begin a new topic: **Energy-Based Models (EBMs)**. To understand these, we will first review the **Ising Model**, as we will rely heavily on these concepts later.

In previous sessions, we focused on learning functions (y given input x). Now, we want to learn distributions of variables y given some parameters θ. As physicists, we prefer to work with energies rather than probability densities. We write this distribution as:

$$P(y|\theta) = \frac{1}{Z(\theta)} e^{-\beta H(y;\theta)}$$

- $Z(\theta)$: The normalization factor, known as the **partition function**. It contains the thermodynamics of the system and allows us to generate averages over variables.
- β: The inverse temperature.
- H: The **Hamiltonian**, which defines the model in terms of interactions between nodes.

2.1.1 Goals of Energy-Based Models

There are two main applications we will focus on:

1. **Associative Memory (Storage)**: Studying how networks store and retrieve data points. For example, if you are shown part of a picture, the network should "remember" and complete the rest. This is similar to a magnetic system where lining up a few spins causes the rest to follow.
2. **Generative Models**: Learning the underlying features that generate data distributions. The goal is not to remember individual points (which is overfitting), but to generate new realizations that look like the data (e.g., generating synthetic faces).

—

2.2 The Ising Model

The Ising Model consists of a regular grid where each point has a binary variable $\sigma_i \in \{+1, -1\}$. The Hamiltonian is defined as:

$$H = -\frac{1}{2}\sum_{i,j} J_{ij}\sigma_i\sigma_j - \sum_i h_i\sigma_i$$

Where J_{ij} represents the interaction between nodes i and j, and h_i is an external field. In a **ferromagnetic** model, $J > 0$, meaning the energy is lower when spins are aligned.

2.2.1 The Gibbs Ensemble and Thermodynamics

The statistics are described by the Gibbs distribution. The partition function Z is calculated by summing over all possible configurations:

$$Z = \sum_{\{\sigma\}} e^{-\beta H}$$

In the thermodynamic limit ($N \to \infty$), the Gibbs distribution can become non-unique. This occurs during a **phase transition**. Below a critical temperature, the system can be in an "all up" or "all down" state, separated by an energy barrier that scales with the system size N.

2.2.2 Mean Field Theory and Hubbard-Stratonovich

To study these phases, we introduce an order parameter: the **magnetization (m)**. We use the **Hubbard-Stratonovich transformation** to move from the discrete variables σ to conjugate variables m.

By integrating out the σ variables, we derive the **Mean Field Free Energy** ($f(m)$). For a system where each spin interacts with many others (valid in dimensions $d > 4$), the free energy per node is:

$$f(m) = \frac{1}{2}\tilde{J}m^2 - \beta^{-1}\log\cosh(\beta(m+h))$$

Setting $\frac{df}{dm} = 0$ gives the self-consistency equation:

$$\tilde{J}m = \tanh(\beta(m+h))$$

The critical point occurs when the slopes of both sides are equal, specifically at $\tilde{J} = \beta$.

2.2.3 Transitions and Susceptibility

- **Second-Order Transition**: At $h = 0$, the magnetization m changes continuously as temperature T crosses the critical point.
- **First-Order Transition**: Occurs when crossing the transition line by changing h. The magnetization jumps discontinuously (e.g., from positive to negative), requiring a release of "latent heat."
- **Susceptibility (χ)**: Defined as $\frac{dm}{dh}$. It measures how responsive the system is to an external field. At the critical point, the curvature of the free energy becomes flat, causing the susceptibility (and fluctuations) to diverge toward infinity.

—

2.3 The Hopfield Model

The Hopfield Model generalizes the Ising model to a dense network where, in principle, every node can connect to every other node.

$$H = -\frac{1}{2}\sum_{i,j} J_{ij}\sigma_i\sigma_j$$

2.3.1 Hebbian Learning

To store P data points (patterns) σ^α, we define the interactions using the empirical covariance, known as **Hebbian Learning**:

$$J_{ij} = \frac{1}{P}\sum_{\alpha=1}^{P} \sigma_i^\alpha \sigma_j^\alpha$$

This ensures that the stored data points σ^α are global minima of the energy landscape. However, this method can also create **spurious minima** (combinations of patterns).

2.3.2 The Mattis Model

If we generalize the binary data to continuous patterns ξ, we can write a more general Hamiltonian:

$$H = -\frac{1}{2P}\sum_{\alpha}\left(\sum_i \xi_i^\alpha \sigma_i\right)^2 - \frac{1}{\sqrt{NP}}\sum_\alpha h^\alpha \sum_i \xi_i^\alpha \sigma_i$$

Here, the terms are scaled by N (number of nodes) and P (number of patterns) to ensure the Hamiltonian scales naturally with the system size. In the next session, we will use this to calculate the storage capacity of the Hopfield network.

Chapter 3

Byers II Module

3.1 Lecture 5 - Differential geometry in ML

3.1.1 Introductory example

When we think about mathematics, it has value in that we sort of can have very clearly defined true propositions. So we could say I can certainly write down some true propositions such as tautological things like 1=1, I could say obviously 2+2=4. And then I could say, oh, well, there's some obviously um counter examples. I could say it is false to say 1=100 or something, that doesn't make any sense. I can do 2+2=5, and we would all say, well, these are falsehoods. Right? Trivial.

But actually let me ask you, are there any ways that I could make the things on the false side true? What makes—what could have I assumed over here that I'm not telling you about maybe over here, or I assumed different things over here that were different from what was assumed there to make these true statements? Anybody have any idea? Sure.

```
True                        False

1 = 1                      1 = 100
2+2 = 4                    2 + 2 = 5
```

Think of it from a physics standpoint. As physicists, why can I make the stuff on the right true, whereas a mathematician will be upset regardless, usually.

'If they have large enough error bars, the result might be true.'

Yeah, that's right. You could have errors and uncertainties. And in fact, on this bottom one here, that's what I could say, like this number and this number are drawn from a Gaussian. Maybe it's centered at 2 and has some sigma that has some width to it. And now it is actually possible for me to draw two numbers from a Gaussian and add them together and there is a way for me to reach five or—or within some bound of five, right? So 2+2=5 within the framework of probability and uncertainty is completely possible. Okay?

So what did it mean over here when I wrote 2+2=4 and thought it was a truth? What was I assuming about those numbers that we did not assume over there? Infinite precision? Right, we were assuming that the sigma here was implicitly going to zero. And so these numbers as real numbers were defined exactly. **In other words, these numbers had an infinite amount of information in them.**

And if that sounds weird and possibly bad, you're right. Okay? Every number you have, you probably learned this in your undergraduate physics lab, you never report a single number, you

always report two numbers. You say, well, here is essentially something like the mean and here is essentially something like the standard deviation of the variance.

Now when you do mathematics and you represent numbers in the way mathematicians do, they tend not to think about that aspect of the number. But as physicists, natural scientists, people working with data in the world, we have to understand that uncertainty quantification is a very important part of our problem. And things that someone might say, well, that is a falsehood, aren't in the right context. Okay, so you got the slightly more complicated one right first. So how do I fix this one?

'Units.'

It's pretty silly, right? But it's not when you realize units matter. So when I use numbers in the world, I have to think about the units associated with them. Not just the uncertainties—that's important too—but the units. And imagine when I say I have to form a dimensionless constant and relate that to other dimensionless constants. In other words, I'm—I'm saying that for units that have dimensions and I have to combine them together in a way that perhaps the dimensions don't matter because they're arbitrary. You make up the dimensions.

So anything you can make up, the world doesn't really care about. It has to be invariant to those choices. So there are certain symmetries inside of the representations of numbers that we use in physics to represent the world that we have to account for and understand. So one meter of course is 100 centimeters. It's trivial, you know, to write it like this. It looks like a trick, but it—it suggests an important change of your mindset to realize units, dimensionality, uncertainty quantification change what you think is true and false in a very strict mathematical sense. It opens your mindset up. And that's exactly what differential geometry does on a wider scale. Okay?

3.1.2 Differential geometry

I promise this is not a philosophical lecture. But let—I want to get our conceptual positioning correct. Okay? And so, this is a famous line about language. The famous philosopher Wittgenstein at Cambridge many years ago said:

```
'Philosophy is a battle against the bewitchment of our minds by language.'
```

Okay? And this is just saying this like you start to think the words are real and you project reality onto the representation and you can get lost in the representation.

Well, that is actually exactly what happened to physics prior to relativity. Physics became lost in the representations of the physics and didn't figure out how to abstract itself from those representations. And so, modifying what Wittgenstein said about um language, we can say that differential geometry is a battle against the bewitchment of our minds by coordinates.

So these coordinate systems, right? And it's important that we need coordinate systems, they are not the enemy of what we are doing, they are a vital tool to have coordinate systems and use them. But it is vital that you not create properties that are inside your coordinate system because you get to make up your coordinate system, right? You choose the coordinate system. You choose to switch coordinate systems.

Now what happens if the physics changed with your choice of coordinate system? This is a disaster. This means physics is not about the world, it's about stuff you choose in your head. And that's crazy.

And this is what Einstein was really battling with when he was trying to create relativity. The challenge of relativity was that, and certainly special relativity stayed fairly trapped within its

coordinate systems. Okay, you could think in terms of essentially Cartesian coordinates and do all of special relativity. But when Einstein attempted to generalize special relativity to general relativity and say, well, now we're going to include um forces and more rigorously and things like gravity, a very distinct problem emerged is that they—you're trapped inside choices of coordinate systems and struggling to figure out what is the coordinate system artifacts and what is the physics that lives beyond the coordinate system. Okay?

You see the difference, right? There's many things you would calculate that you'd say: 'If I change coordinates, does this change too?' And does that change get reflected in the mathematics? So we have to be careful. Coordinate systems are very much like these choices of units or understanding whether the numbers we have have finite precision. **Coordinate systems are also something that we assume and can build in artifacts and mistakes into our representations.** And I hope you start to see a hint of why this might matter in machine learning or data analysis.

3.1.3 Recap: ML turns probabilistic problems into geometrical problems

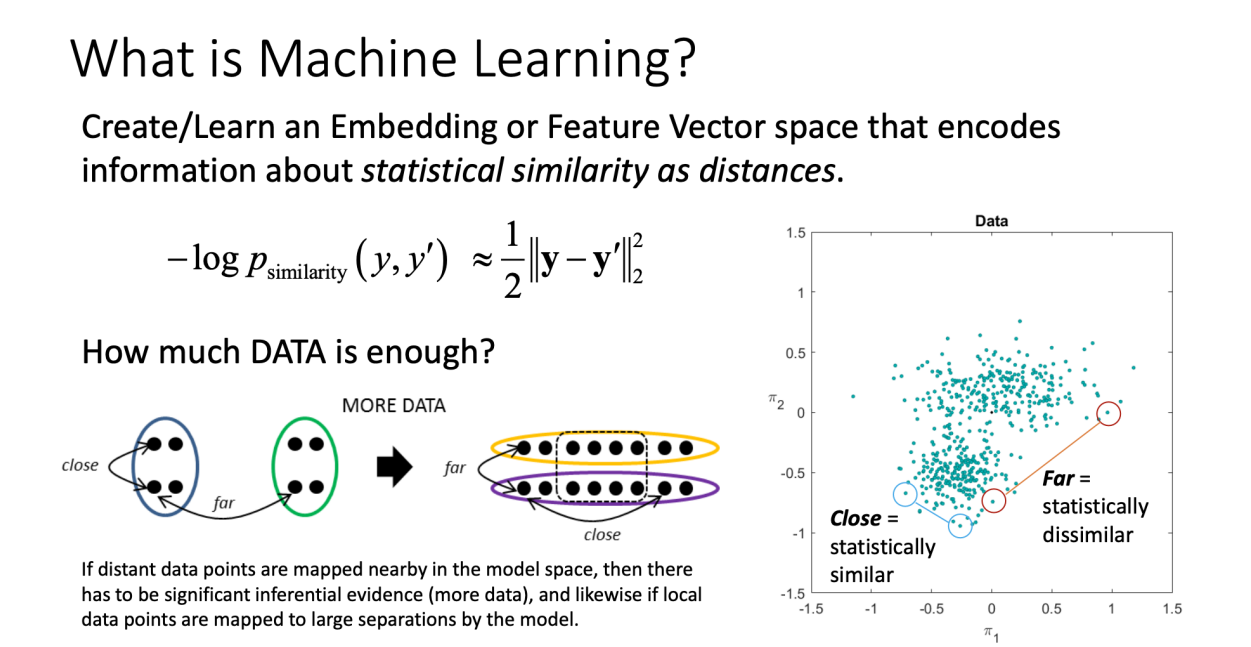


Figure 3.1: The self information potential $p_{similarity}(\mathbf{y}, \mathbf{y}')$ is the fundamental quantity

So let's remind ourselves. I—if you recall back,I was trying to say: 'Well, what is machine learning?'

So you've been exposed to information theory—a very peculiar science, a very confusing in its own right. It always seems to have the answers but never tells you them. It's like an oracle. And I'll come to why information theory is later in another lecture—why information theory is so problematic. And it has to do with the fact that in the world you never have the probability distributions, you have the data. And information theory works perfectly if you have the probability distributions, but it doesn't work so well if you just have the data. And I can show you how to try

to fix that so that you can use information theory in a practical sense with data, which is something evidently they never want to talk about in information theory courses. But anyway.

So this kind of information theoretic term, which is sometimes called, if you remember, the **self-information**. **I claimed in our discussion this is the smarter object to look at in many cases.** The probability itself is a little strange to think about. But this term, $-\log$, is actually more intuitive, especially for physicists, because this is essentially like a potential energy. **And it's the potential energy in which sits your data. And so this is the constraints that keeps your data organized in the space as if they were traveling particles doing Brownian motion traveling around in a potential.**

Now what does machine learning do? Machine learning says: 'Okay, I'm given this potential.' In fact, I would say any good physicist would probably do this too—Taylor expand it! Right? Let's expand this term and pick up the lead order term. Okay, the first non-trivial lead order term is this one where we say I've essentially expanded around some location and it's—it's like a harmonic oscillator, right? So from a physics standpoint, you should be like 'Sure.' This is a—a good starting point. The—it's like saying I have my potential that's holding my data together—let's imagine it's initially like a harmonic oscillator potential as the lead order non-trivial term of this expansion.

And so this is an approximation and you say: 'Ah, great. I can take this probability of similarity.' This is where the interpretive part of the problem comes in.

So what's happening is that the probability of similarity says that you give me an object, and you give me another object, and I have some measure of what I can see where I say: 'Are these objects similar or not with however I see them?' Okay. Um, if you remember in one of my earlier lectures, I called that **relevance**. What is the relevant thing I can see that's different? Because the brutal fact about all the data in the world is pretty much it's all different.

Every—you could tell almost—I could tell every one of these tiles apart from each other, right? Every one of these is unique in some way and they could be uniquely identified. All the tiles in all the buildings in all the world. Now the question is: 'In what task would that be relevant? Would it matter that I could tell every tile apart?' Now there are tasks where it matters that I could tell your fingerprints apart, right? Or your facial features, right?

So knowing that every one of you is different is important for some tasks. Other tasks I say: 'I detect a human, I detect a tile.' That is a sufficient level of classification. So it's relevant what task I do.

So when I say similarity here, I'm saying similarity with respect to doing something. A choice about what you do. So you can almost always with very sophisticated visual systems like we have, you can always see difference. The question is: 'You have to see the difference and figure out which differences matter.' And that's what a classifier is. A classifier is figuring out the differences that matter that you can see. If you can't see them, it's obviously hopeless, you can't use those features.

You want to say the distance—remember, machine learning is saying: 'I want to—probability's hard to work with. It's complicated computationally.' And what's—what's easy to work with? Remember, what is the—one of the fastest things we can do on computers? Numerical linear algebra. Okay? If you can convert a nasty probability problem into a numerical linear algebra problem, it's a big win—a very, very big win for you. And that's what machine learning exploits. It's one of its key differences with statistics, and why it came more from computer science. Computer scientists looked at large data sets and said: 'We will have to have computational efficiency,' and so how do you leverage things like numerical linear algebra?

So now we took a problem that was over in a probability space with a probability rules, which are actually quite inconvenient often to work through (you know, they're only two rules, right? in

terms of marginalization and completeness and things, but it actually creates a kind of calculus mess over there). And so here, we said: 'Now it's just distance.' And then what we did was—see the script Y and the—the bold-faced Y—is somehow—I mean, what—if I tell you I'm going to do distances between things, what does that mean?

Well, it means they're **vectors**. They're like points in a space, like a vector space. And I say: 'Well, there's this rule, this Euclidean rule of take the coordinate'—and I'll say 'coordinate' with 'scare quotes' because of our differential geometry preamble—and we say: 'Okay, what is the distance between this point and this point just through the usual Euclidean way of subtracting the components-wise differences between each of those, and then we can add them up.'

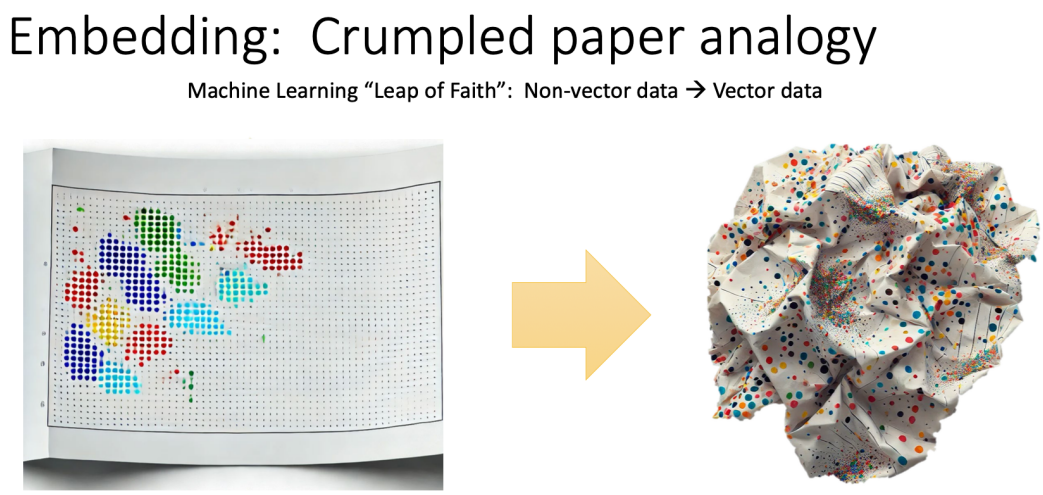
And so now we have a—a way of saying: 'Ah, I can create a space that this is in, my data, where I've mapped the things themselves,' because **I'm sure you know tiles and pieces of chairs are not vectors. Right? But I can figure out funny ways to convert these into vectors**. Just like you do with a large language model turns bits of your speech and words into vectors and then processes on the vectors. All right? And **this is the embedding. And the embedding is the key step**. The vector space embedding of machine learning is critical. It's one of the more powerful aspects of what's going on, if it's done correctly.

And so now I can examine the distances between things and say: 'Ah, closeness in the space of features is um statistical similarity,' okay? 'And separation in that space represents statistical dissimilarity.' **But always remember: This was done with respect to something.** There was a task of some kind I was doing. **All that information is hidden inside what did you mean by the probability of similarity that was over there**.

All right. Now, this is reviewing what I talked about before, but I think it's actually—it can be a kind of confusing concept. And so, hopefully it's more clear the second time, in person, going through it. So I'm just going to pause a second and see if there are any objections, questions, anything about it or you're like: 'Hey, it's pretty clear what this is right now and why it's the entry point into the representations of machine learning.' Okay?

Machine learning creates a geometry, but your statistical problems live more in a space of probability. And so we've got to get them out of there, because these are terribly hard problems over there. And now we're going to turn into these geometry problems with all sorts of crazy tricks, and then we'll transform them back out of the geometry back into the space of the predictions, you know, to predict the next token in ChatGPT .

3.1.4 Recap: the embedding and the crumpled paper analogy



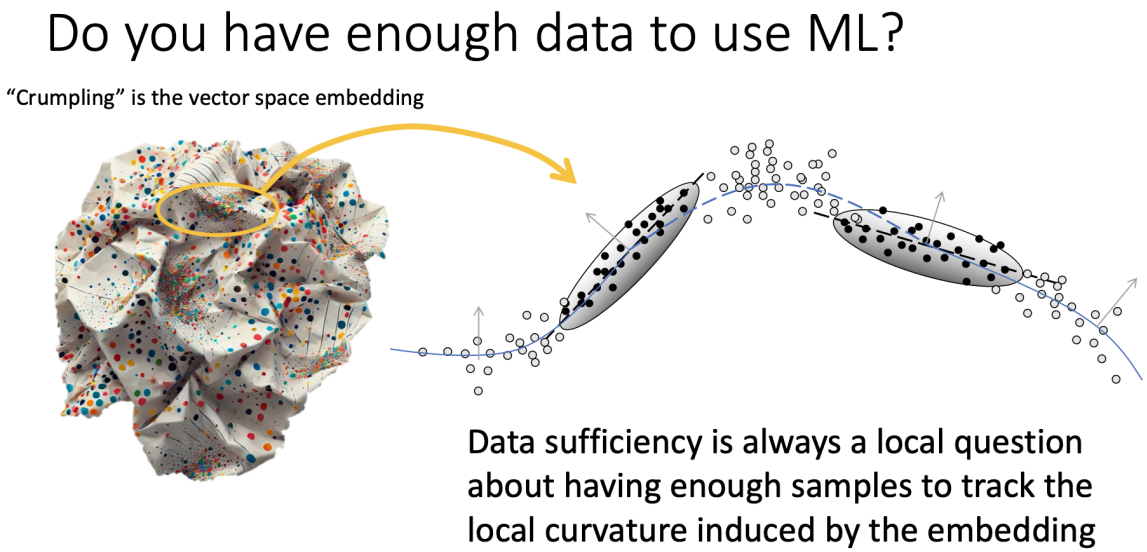
And so it's like you've crumpled up a piece of paper I cannot go anywhere in the data and say: 'Oh, I believe *this*

$$-\log p_{similarity}(\mathbf{y}, \mathbf{y}') \sim ||\mathbf{y} - \mathbf{y}'||^2 \tag{3.1}$$

is true.' In other words, then it's just a giant ball with Euclidean measures everywhere. This is not the case. This is only true locally. Okay? And you might—you might very correctly ask the question: 'Well, what the hell does that mean? Locally. What's local? How do I figure out what local is?' It turns out that's kind of the real problem you're solving—is what does local mean?

'Do I have enough data that I can believe in *this*?. *This* is more of a belief that you can do this. Okay? It's sort of an ansatz or, you know, a postulate that you can do it. So it's just a normalization trick in machine learning when they do that. But it's like: 'Do I have enough data to believe that I can make this true?' And if I don't, I'll just find out I just have this high-dimensional noise ball that I can never quite figure out what's going on with.

3.1.5 Data sufficiency is a *local* question



So what's the other thing they almost never talk about in machine learning? How do I figure out if I have enough data? Did anyone ever explain that to you? How do you have enough data in machine learning? They have no idea.

But I'm saying now you actually have a way to think about this, and it's related to the differential geometry that we are coming to, which is essentially the closeness or the farness compared to the curvature of the thing.

So if the space is flat, then it probably doesn't take many points of data to sort of figure that out, right? So I have something here that's relatively flat sitting inside some space and maybe this thing here is lower dimensional than the space it's sitting in. Oops. Okay? But it's flat. If I believe it's flat, that's easy, right? Just a few points. But then this rule here applies everywhere in that space. You can use this all over this entire space. There's no problem.

Now the problem starts... now we've got a different sort of space. So we have something that's more like a manifold and it may be inside of RD, but then we have a question like: 'Okay, well in some sense that we'll talk about, the dimension of this might be little d, like this up here,' but we have, you know, curvature, right? And so how many points do I need to learn this object versus this object?

More. **This is a more complicated object. If you want to be more formal about it, it takes more parameters to identify this object than the number of parameters it takes to identify that object.**

So on this object, *this [Euclidean distance]* is not true everywhere. But where is it true? Tangent spaces.

And that turns out to be what the essence of differential geometry is. It's very complicated objects embedded in a space, but the little pieces of the objects are like the Euclidean space you know and love. It turns out **manifolds are the most general place that you can still do**

calculus. Those are the most general places that calculus is still functional in the form that you think of it from an undergraduate standpoint.

Well, I've worked 20 years in machine learning, and the first question everyone always asks is, you know, "I have X amount of data, is it enough?". I've no idea, right? How could I possibly know the answer? **Because it's not the amount of data you have, it's where the data is**

So that's why you can't answer the question 'Is it enough data?' I have to say 'Given your embedding and understanding the amount of curvature in it, is the data placed correctly to find the structure that's hiding inside the embedding space?' All right. It just comes with the territory when **you say 'I turned my probability problem into a geometry problem' that, you know, nothing's quite for free. There's going to be some trouble that lives inside the other space, and curvature is kind of that trouble.**

So we've crumpled our paper — **and remember, in your data, you just have the points here, you don't have the paper. If you had the paper, your job would be easy** — just unfold the paper.

But the paper doesn't exist. **You have to infer the paper from the points**. But notice the trouble here is when the paper got crumpled, there's lots of points that became very close to each other, but they're on different pieces of the paper. I have to be able to figure out the difference, by following the underlying structure. The paper is the manifold. Okay?

And all I have is a point cloud. I have no manifold. **The manifold is a model concept. The data is what you have**. So, as I've already suggested here, we have a—a tool that we'll try to think about using—tangent spaces. And because on those tangent spaces, that's where the geometry actually works—on the tangent space. And so we have to be able to have the points to follow this around and then be able to say: 'Well, **how do I learn these tangent spaces?**

So that's the—the primary connection between differential geometry and machine learning. They both are—they—the machine learning induces a geometry from its probabilistic kind of problem, it induces this geometry, but then that geometry um has to be learned from just a point cloud. It's just a bunch of points in a space, there's no topology to it, there's nothing, it's just points.

```
QUESTION: "how lower dimensional are tangent spaces?'
'Is there a rule of thumb?'
```

No, I don't—but think of it this way. Think of the generative process that makes the data. And then ask how many parameters do you believe in are needed to be specified to make that data. That is actually the dimensionality of the manifold the image sits on, in a sense. So when you look at this you go where is the real world? Where is the stuff I'm trying to understand versus where are my models of the stuff? Well, think of it like I have a data space here. And what happens is that sensors create giant data spaces. And you might say well that's inconvenient. We wish sensors didn't create giant data spaces for us. But that's not true. The sensor creates this giant data space R_D. Like think of a camera. So I have a camera, you know, that's like 1920 by 1080 pixels, you know like HD, three color depths... it's like 6 million you know numbers to represent that. You're like wow that is... that's a 6 million dimensional space if you want to think of it that way.

But remember very, very, very, very little of that space is actually used. Right? The space of natural images is like a wispy tiny thing hidden inside of that space that occupies very, very little of the actual space of all the dimensions that the sensor—the HD sensor—could actually see. And that's an important fact because you learn more from a sensor that could have seen anything but only saw this. That means that was like a lot of information content. Those choices—of what the sensor actually saw versus what it could have seen—is an incredible amount of information.

So, the downside of it is you're... you're stuck though in this data space that's huge. Think of it like in statistical mechanics, like a phase space or something for dynamics. It can be quite huge, right? Six to the n number of particles. So it's this massive space over there.

But just like in physics and mechanics, you learn that there's only certain restrictions on that of the generative process, whether it's conservation of energy and how momentum works and symmetries like angular momentum. They all reduce the dimensionality and you can actually solve the problem because there's a model space that's lower dimensional so that you are not crushed by the curse of dimensionality.

So have you all heard of the curse of dimensionality? Well, the important thing is to understand where does the curse live. The curse of dimensionality is never in your data. If your data has a curse of dimensionality, you're screwed. There's nothing you can do. It's over. Give up. There's nothing to fix it. Okay? Curses of dimensionality happen in your model. Your model has lots of degrees of freedom you have to prune and control and manage. And so the curse of dimensionality in your model space though is about regularization, right? You've heard regularization, or having a Bayesian prior, you know, or something. Like the log of the... of the Bayesian prior is sort of like the regularization in your neural network or your... your machine learning problem.

And so over here in the model space you say well there's some sort of restriction that's going on here, and in large measure that's that dimensionality reduction we're talking about. How do we find that tangent space? How do we get down onto that manifold? How do we find the subspaces where that data really is living? Because those subspaces represent the fact that just in an example I was explaining to you about like the image of something, there's not that many parameters needed in the generation of the data, right? The sensor—the... the camera sensor—may have 6 million dimensions but I don't need 6 million dimensions—6 million parameters—to create the image. I may only need 50 parameters or 100 parameters. And that's the lower dimensional space that we're looking for here.

3.1.6 Locality and data attribution: Neural networks versus Kernel Methods

Geometric Partitioning Schemes

A. Cluster-Based (Local Patches)
Assumption:
- Data concentrates in localized regions.

Pros:
- Clear attribution.
- Local uncertainty.
- Honest ignorance in sparse regions.

Cons:
- Complexity grows with volume.
- Weak extrapolation.

B. Low-Dimensional Subspace (Global)
Assumption:
- Data lies near a single linear manifold.

Pros:
- Parameter-efficient.
- Global compression.

Cons:
- Fragile to curvature.
- Diffuse attribution.
- Strong global prior.

C. Hyperplane Partitions
Assumption:
- Data separable by half-spaces.

Pros:
- Compact binary encoding.
- Decision-focused.

Cons:
- Global combinatorial complexity.
- Brittle boundaries.

Local Models	Global Models
Scale with data density	Share parameters globally
Honest uncertainty	Smooth extrapolation
Harder compression	Efficient compression

Question: Is failure to extrapolate a bug or a feature? In science, often a feature.

A neural network is a global fitting scheme, not a local fitting scheme. So the neural network says: 'No, we're going to solve a problem this way.' It's going to chop the space up into half spaces essentially and talk about being on either side of those and it's almost like a zip code. To specify your position, you'll say you are on, you know, this side of that and that side of this and you'll sort of specify yourself essentially on the different sides of all these hyperplanes.

But notice what that means. And this is why neural networks are very bad for experimental science. Can you tell me where I should get more data? Like I go and I do my project and I find out unfortunately for whatever reason my analysis doesn't really come through with the data I have, but now I would like to do some experimental design. I would like to get more data. And well the question is: 'Well, where would it matter to get more data? Where in this space do you need more data?'

Well, if you're over here in this picture, where do I need more data? It's pretty clear. I'm tracking this curvature, I may not have enough data there and my local system here, like I didn't have enough points and I say 'Ah, I can't resolve this. It's just a ball. And this is just a noise ball here. I did okay here, not good there.' So from an experimental design standpoint, I know where to go.

Now I'm over here with my neural network. How does it tell me where to add the points? I don't know. Because it's every one of these pieces of the model. **It's a global cut across the entire data space.** So it's very hard for me to do um assignment of data, to say *what* about my data created *what* about my model.

I would claim you guys should be uncomfortable, somewhat unhappy with that approach and cautious. And the caution is that if I can't do data attribution and I can't do data sufficiency, well, you know what the next thing that I'm facing is? If I can't do those things, I can't do interpretation.

In things like, you know, physics and chemistry and biology and the things we do, we're not looking for inscrutable black box, we want interpretability even if you have pretty good perfor-

mance in the algorithm. So neural networks are very unfriendly to interpretability because of these problems with data attribution and data sufficiency.

It's a geometric choice of how a neural network is constructed that creates this type of problem.

```
QUESTION: "'Could you maybe explain why the neural network is like slicing the space?
Because I mean if we have access to the slices then we could like define the density,
and then we would know when we need data, right?
 Like the amount of data we have per space and if the density is low.'
```

It's this vector W points in a place. The position of this plane is set by this bias piece right here. So think of it as like—let's pick... I don't have colored chalk, so I apologize for that, I'll have to work with white. I'll—let me make this double arrow thing like that. This double arrow is really saying what—where is it through the transformation of the weights, right? So I'm saying this is kind of like this arrow and then this is sort of setting well along this line where do I cut it to put the plane where the variation of the plane—now it's not—I take your point, it's not strictly like oh there's this plane sitting there.

It's more like a plane that's blurred by the scale of the parameters of W. So it could become very blurred if the values here um become very large or it becomes very sharp. This allows you to do that. But what I'm pointing out is you—I mean you've seen this, this is the typical neural unit in your neural network, it's written this way, right? for a layer. And so this is the input layer, that's the output layer, so that's the input, that's the output.

And so now we say: 'Here I am on the one layer and then I come out the other layer,' but it's kind of this object that gives the game away—the dot. Sigma dot. This is coordinateized. So you go into the coordinates of that thing and you pick off the first coordinate and use it. And then you pick off the second one and use it.

So it's kind of like I line it up with something and then I cut, and I say all the data that's—figures out how to set the W and the B is the data on either side of where this arrow decided to point and where I would put the B. And so all the data on either side in principle, probably mostly near the boundary here, right here, that decides what happens here. So that cutting across the space is what creates the problem.

In other words, it's not like oh there's just a certain compact set that is controlling what happens to this piece of a model. The volumes involved here are essentially infinite.

If there were a machine learning professor sitting in here, they would say: 'Oh, you're just trying to resurrect kernel methods over here.' **A kernel is in general a compact object that localizes what data is involved in the model. So the kernel approach is sort of like the other road to travel.** There's the neural network one, and then there's sort of the kernel-based approaches. And so I'm saying that the mathematics of understanding spaces is more aligned with kernels and those techniques than they are to neural networks. Even though machine learning and everything is mixed together and you can have kernels that live in neural networks in weird ways.

3.1.7 We need to learn how to do *local* PCA

But the difference is that PCA just does it on everything. It's kind of dumb. It just takes the entire data set and says fine, we'll just do this spectral... the singular value decomposition on it. We'll find the singular values that are bigger, we'll keep those, we'll throw away the smaller ones,

and that creates a projection matrix and then creates a subspace. And the projection matrix takes the data and puts it down on something like this tangent space.

The problem is that if you did this for the entire data set, I mean what's going to happen? I mean it's kind of the wrong answer for all the data. It'll project it down but it'll destroy a lot of the intricate structure of your data. So the trick is: how do you do PCA in a slightly more intelligent way with a subset of the data? Okay?

But it's not, but fundamentally it's no different than what you did with PCA when you just did it to all the data and just kind of projected everything down. Now granted, if I had this example over here, PCA is fine. If I do PCA on that object globally, it's no big deal. It'll find exactly what you need. It's a linear object, no problem. But if I do PCA on an object that has curvature in it, I could have a serious problem because the projection isn't really capturing the geometry of the object. And from a machine learning standpoint, it's not preserving that Euclidean distance effect that we were counting on to be able to do things. Okay.